

Python

Summary Session 2



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Какими типами в Python представлены числа?

?

Что такое арифметические операции?

?

Какие бывают арифметические операции?



Арифметические операции

Python поддерживает стандартные арифметические операции, такие как сложение, вычитание, умножение и деление.

Полезно знать



Операция

```
print("Сложение:", "3 + 5 =", 3 + 5)
print("Вычитание:", "7 - 2 =", 7 - 2.0)
print("Умножение:", "4 * 6 =", 4 * 6)
```

Деление

```
print("Деление:", "8 / 2 =", 8 / 2)
print("Целочисленное деление:", "7 // 2",
      "=", 7 // 2)
print("Остаток от деления:", "7 % 3 =",
      7 % 3)
```

Полезно знать



Переменная

Если какой-то результат в последующем нужно использовать повторно, то можно поместить его в переменную.

```
sum1 = 3 + 5
a = sum1 + 1
b = sum1 * 3
print(a, b)
```

Повторение

Если какое-то действие нужно совершить один раз, то хранить информацию в переменной бессмысленно.

```
print(3 + 5)
```


Экспресс-опрос

?

Что такое ZeroDivisionError?

?

Когда возникает ZeroDivisionError?

?

Какие общие правила приоритета операций?



ZeroDivisionError

Это исключение, которое возникает когда происходит попытка деления на ноль.

Когда возникает ZeroDivisionError?



Пример

- При делении целых чисел на ноль
- При делении вещественных чисел на ноль
- При использовании оператора остатка от деления на ноль

Код

```
a = 10
b = 0
result = a / b
```



Скобки

В Python, как и в математике, операции выполняются в определенном порядке. Это называется приоритетом операций.

Общие правила приоритета операций:



Скобки (())

Операции внутри скобок всегда выполняются первыми, независимо от их приоритета.

Пример

```
result = (2 + 3) * 4 # Скобки  
заставляют сложить 2 и 3 перед  
умножением
```

```
print(result) # Результат: 20
```

Общие правила приоритета операций:



Возведение в степень (**)

После скобок, возведение в степень имеет наивысший приоритет.

Пример

```
result = 2 * 3 ** 2  
# Это интерпретируется как 2 * (3 ** 2)  
print(result)           # Результат: 18
```

Операции имеют одинаковый приоритет

1

Скобки ()

2

Возведение в степень **

3

Умножение, Деление, Целочисленное деление, Остаток от деления *, /, //, %

4

Сложение и Вычитание +, -

Экспресс-опрос

?

Что такое неизменяемые типы данных?

?

Приведите примеры неизменяемых типов данных.



Неизменяемые типы данных

В Python существуют неизменяемые (immutable) и изменяемые (mutable) типы данных. Неизменяемые типы данных не могут быть изменены после создания объекта, а изменяемые могут.

Неизменяемые типы данных



Пояснение

То есть если у нас есть переменная `num` равная 5, то мы не можем изменить значение этой переменной, например с помощью арифметической операции:

Пример

```
num = 5
num + 2          # В переменной num
останется значение 5
print(num)
```

Неизменяемые типы данных



Пояснение

Для того чтобы изменить значение в num нужно присвоить переменной новое значение.

Пример

```
num = 5
num = num + 2      # В переменную num
будет присвоен результат вычисления 5 +
2
print(num)
```

Экспресс-опрос

?

Что такое Операторы приращения?

?

Чем оператор `(+=)` отличается от `(*=)`?



Операторы приращения

Это операторы, которые изменяют значение переменной на основе её текущего значения.

Операторы приращения



Прибавление (+=)

```
a = 5
a += 1 # Эквивалентно: a = a + 1
print(a) # Результат: 6
```

Умножение (*=)

```
a = 5
a *= 2 # Эквивалентно: a = a * 2
print(a) # Результат: 10
```

Экспресс-опрос

?

Что такое Множественное присваивание?

?

Как производится обмен значениями между переменными?



Множественное присваивание

Это особенность Python, которая позволяет присваивать значения нескольким переменным одновременно в одной строке.

Присваивание нескольких значений



```
a, b, c = 1, 2, 3 # Инициализируем переменные a, b, c значениями 1, 2, 3 соответственно
print(a, b, c)   # Вывод: 1 2 3
```

Присваивание одинакового значения нескольким переменным



```
a = b = c = 0      # Инициализируем каждую из переменных a, b, c значением 0
print(a, b, c)     # Вывод: 0 0 0
```

Обмен значениями между переменными



```
a, b = 5, 10
a, b = b, a # Меняем местами значения a и b
print(a, b) # Вывод: 10 5
```

Экспресс-опрос

?

Что такое Преобразование типов?

?

Чем явное преобразование отличается от неявного?



Преобразование типов

Это процесс преобразования значения одного типа данных в значение другого типа.

Преобразование типов



```
num1 = "10"  
num2 = "5.0"  
print(num1 + num2)  # Конкатенация строк  
print(num1 / num2)  # Попытка выполнить деление строк
```



Преобразование типов

При попытке выполнить деление `num1` на `num2`, Python вызывает ошибку `TypeError`, поскольку оператор `/` не поддерживается для строк.

Преобразование типов



```
num1 = "10"
num2 = "5.0"
result1 = int(num1) + float(num2) # Приведение строк к числовым типам и выполнение сложения
result2 = int(num1) / float(num2) # Приведение строк к числовым типам и выполнение деления
print(result)                    # Вывод: 15.0 2.0
```




Явное преобразование

Происходит, когда разработчик вручную преобразует один тип данных в другой с помощью встроенных функций.

Целое число в строку



```
a = 5
b = str(a)      # Преобразуем целое число в строку
print(b)        # Вывод: '5'
print(type(b))  # Вывод: <class 'str'>
```



Неявное преобразование

Python может автоматически преобразовывать некоторые типы данных в других контекстах, например, при выполнении арифметических операций между числами разных типов.

Неявное преобразование



```
result = 5
print(type(result))    # <class 'int'>
result += 3.14         # Неявное преобразование int в float
print(type(result))    # <class 'float'>
```

Преобразование строки в число



```
a = "10"
b = int(a)    # Преобразуем строку в целое число
print(b)     # Вывод: 10
print(type(b)) # Вывод: <class 'int'>
```

Преобразование вещественного числа в целое



```
a = 10.7
b = int(a) # Преобразуем вещественное число в целое
print(b) # Вывод: 10 (дробная часть отбрасывается)
```

Преобразование разных типов



```
a = "abc"
b = int(a) # Ошибка: ValueError
```

Не все типы могут быть преобразованы друг в друга

Преобразование вещественных чисел



```
a = 5.99
b = int(a)
print(b) # Вывод: 5
```

При преобразовании вещественных чисел в целые дробная часть отбрасывается

Преобразование строк



```
a = "False"
b = bool(a)
print(b) # Вывод: True
```

При преобразовании строк в логический тип любая непустая строка становится True

Экспресс-опрос

?

Что такое ValueError?

?

Какие значения может принять логический тип данных?



ValueError

Это встроенное исключение в Python, которое возникает, когда функция получает аргумент правильного типа, но с некорректным значением.

ValueError



Ответ

Например, если вы попытаетесь преобразовать строку, которая не может быть интерпретирована как число, в целое или вещественное число, будет вызвано исключение `ValueError`.

Пример

```
a = "abc"
b = float(a) # Ошибка: ValueError: invalid
literal for float() with base 10: 'abc'
```



Логический тип данных

Используется для представления двух возможных значений: True (истина) и False (ложь).



True и False

Эти значения помогают программе принимать решения. Например, в зависимости от того, истинное значение или ложное, программа может выполнить разные действия.

Полезно знать



Логические значения

Они могут быть созданы напрямую или быть результатом логических операций, сравнений или приведения типов к **bool**.

Создание напрямую:

isFinished = **True**

hasAccess = **False**

Экспресс-опрос

?

Какой будет результат программы `print(bool(5))`?

?

Какой будет результат программы `print(bool("Hello"))`?

Значения



True

Любое ненулевое число, непустая строка, список, кортеж и т.д.

False

None, 0, пустые последовательности (например, [], {}, "").

Преобразования в bool



Автоматическое

Python автоматически преобразует значения в True или False, когда это необходимо.

При этом Python определяет, является ли значение "истинным" или "ложным", исходя из его типа.

Явное (ручное)

Можно использовать функцию bool.

Автоматическое преобразование



Условия if



Циклы



Логические операции

Явное (ручное) преобразование



```
print(bool(5))          # True (ненулевое число)
print(bool(0))          # False
print(bool(None))       # False
print(bool([]))         # False (пустой список)
print(bool("Hello"))    # True (непустая строка)
```

Экспресс-опрос

?

Что такое Операторы сравнения?

?

Что это за оператор $\geq$?



Операторы сравнения

Эти операторы применяются для проверки равенства, неравенства, а также для сравнения величин (больше, меньше и т.д.).

Оператор	Описание	Пример	Результат
==	Равно	5 == 5	True
!=	Не равно	5 != 5	False
>	Больше	3 > 5	False

Оператор	Описание	Пример	Результат
<	Меньше	$4 < 6$	True
>=	Больше или равно	$5 >= 5$	True
<=	Меньше или равно	$3 <= 2$	False

Операторы сравнения с числами



Оператор == (равно)

Сравнивает два числа на равенство

Пример

```
a = 10
b = 10
print(a == b) # True, так как a равно b
```

Операторы сравнения с числами



Оператор != (не равно):

Проверяет, не равны ли два числа.

Пример

```
a = 10
b = 5
print(a != b) # True, так как a не равно b
```

Операторы сравнения с числами



Оператор > (больше)

Проверяет, больше ли одно число другого.

Пример

```
a = 8
b = 10
print(a > b) # False, так как a меньше b
```

Операторы сравнения с числами



Оператор < (меньше)

Проверяет, меньше ли одно число другого.

Пример

```
a = 5
b = 7
print(a < b) # True, так как a меньше b
```

Операторы сравнения с числами



Оператор `>=` (больше или равно)

Проверяет, больше или равно ли одно число другому.

Пример

```
a = 5
b = 5
print(a >= b) # True, так как a равно b
```

Операторы сравнения с числами



Оператор `<=` (меньше или равно)

Проверяет, меньше или равно ли одно число другому.

Пример

```
a = 3
b = 4
print(a <= b) # True, так как a меньше b
```

Экспресс-опрос

?

Что такое логические операторы?

?

Что такое оператор not?



Логические операторы

Используются для выполнения логических операций над значениями или выражениями.

Оператор	Описание	Пример	Результат
and	Логическое "И": возвращает True, если оба условия истинны	True and False	False
or	Логическое "ИЛИ": возвращает True, если хотя бы одно условие истинно	True or False	True
not	Логическое "НЕ": возвращает противоположное значение	not True	False

Логические операторы



Оператор and (логическое "И")

Оператор and возвращает True, если оба условия истинны.

Если хотя бы одно из условий ложно, результатом будет False.

Пример

```
a = 5  
b = 10
```

Проверяем что оба числа положительные

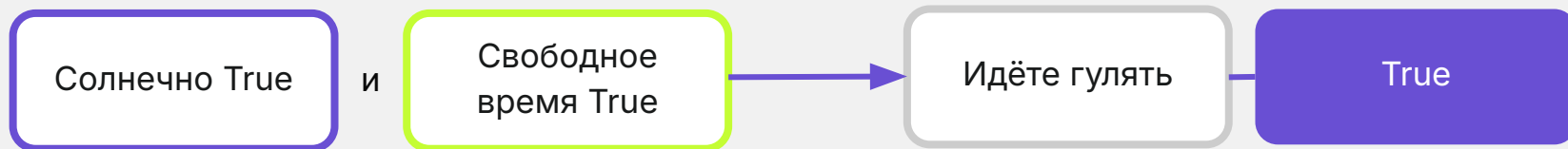
```
print(a > 0 and b > 0) # True, так как оба условия  
истинны
```

Пример из жизни для логического оператора and

Представьте, что вы хотите пойти на прогулку, но только если **солнечно** и у вас есть **свободное время**.

Оба условия должны быть истинными, чтобы вы пошли гулять.





Это как сказать:

"Я пойду гулять, если одновременно и погода хорошая, и у меня есть свободное время."

Логические операторы



Оператор or (логическое "ИЛИ")

Оператор or возвращает True, если хотя бы одно из условий истинно.

Если оба условия ложны, результатом будет False.

Пример

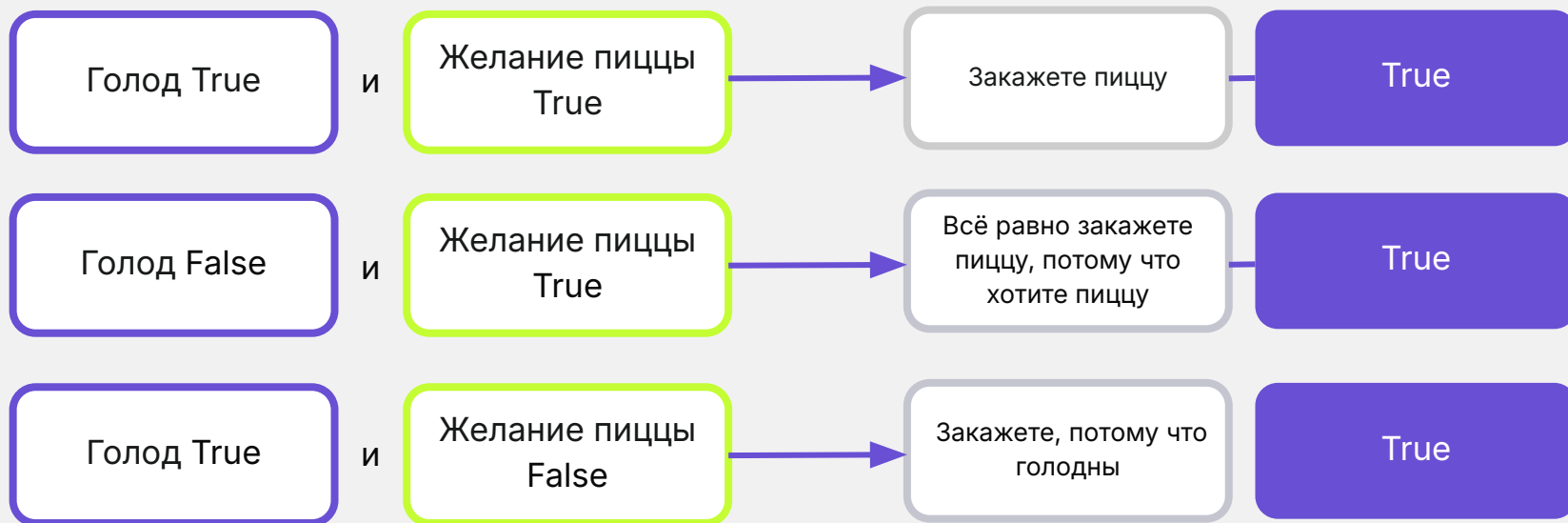
```
a = -5
b = 10
# Проверяем что хотя бы одно из чисел
# положительное
print(a > 0 or b > 0)  # True, так как b > 0 истинно
```

Пример из жизни для логического оператора or

Теперь представьте, что вы хотите заказать пиццу, но вы сделаете заказ, если у вас либо **голод**, либо **желание пиццы**.

В этом случае, достаточно, чтобы хотя бы одно из условий было истинным.





Это как сказать: "Я закажу пиццу, если либо я голоден, либо у меня есть настроение её съесть."

Логические операторы



Оператор not (логическое "НЕ")

Оператор not возвращает противоположное значение.

Если выражение истинно, not превращает его в ложное, и наоборот.

Пример

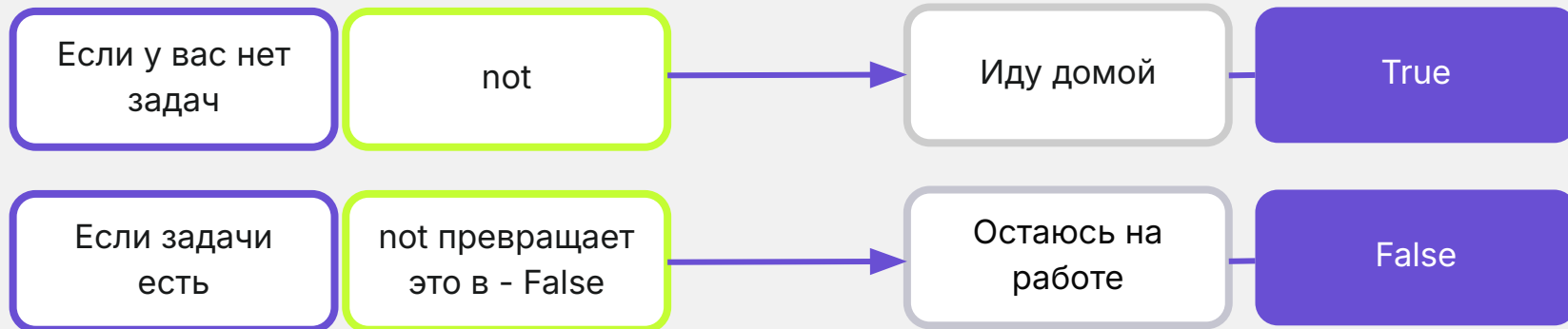
```
a = False
```

```
print(not a) # True, так как это обратное значение от False
```

Пример из жизни для логического оператора **not**

Представьте, что вы на работе и планируете уйти домой. Вы пойдёте домой только в том случае, если у вас нет задач.

Здесь оператор **not** работает как отрицание: если задач нет, то вы идёте домой.





Это как сказать: "Я иду домой, если у меня нет задач", что является логическим отрицанием выполнения работы.

Экспресс-опрос

?

Какой приоритет операторов в Python?

?

Как скобки влияют для приоритета?

Комбинирование



Возможности

Вы можете комбинировать несколько логических операторов в одном выражении для создания более сложных условий.

В этом примере результат будет True, потому что оба числа больше 0 (условие с and истинно), и также `a == 5` (условие с or истинно).

Пример

```
a = 5
```

```
b = 10
```

```
print(a > 0 and b > 0 or a == 5)
```



Приоритет операторов

В Python логические операторы имеют определённый порядок выполнения (приоритет).

Порядок приоритета



1. not (логическое "НЕ") — выполняется первым.
2. and (логическое "И") — выполняется вторым.
3. or (логическое "ИЛИ") — выполняется последним.

Порядок приоритета

Пример:

a = True

b = True

c = True

print(not a or b and c)

В этом выражении сначала выполнится оператор not, затем and, и последним — or:

1. not a or b and c # Подставим значения переменных
2. not True or True and True # not True → False
3. False or True and True # True and True → True
4. False or True # False or True → True
5. True



Скобки для приоритета

Если вы хотите изменить порядок выполнения логических операторов, можно использовать скобки, чтобы явно указать, какие операции должны быть выполнены в первую очередь, так как скобки имеют наивысший приоритет.

Скобки для изменения приоритета

Пример:

result = not (a or b) and c

Теперь оператор or выполняется первым (из-за скобок), затем not, и в последнюю очередь — and:

1. not (a or b) and c # Подставим значения переменных
2. not (**False or True**) and True # False or True → True
3. **not True** and True # not True → False
4. **False and True** # False and True → False
5. **False**

Математические операции и операторы сравнения



Приоритет

Математические операции имеют более высокий приоритет, чем операторы сравнения. Это значит, что сначала Python выполнит все вычисления, а затем сравнит результат.

Пример

```
result = 3 + 2 > 4 # Сначала выполняется  
сложение (3 + 2), затем результат сравнивается  
с 4
```

```
print(result)      # True, так как 5 > 4
```

Пример из жизни для нескольких условий

Использование and и or

Предположим, вы хотите узнать, можно ли пойти гулять.

Условия: (погода хорошая **или** день выходной), **и** у вас есть свободное время.

```
good_weather = True
```

```
is_weekend = False
```

```
free_time = True
```

```
can_go_out = (good_weather or is_weekend) and  
free_time
```

```
print(can_go_out) # Результат: True
```

Пример из жизни для нескольких условий

Использование not с другими операторами

Допустим, вы планируете поехать в отпуск, если у вас **нет** работы **и** достаточно денег.

```
have_work = True
```

```
enough_money = True
```

```
can_go_vacation = not have_work and enough_money
```

```
print(can_go_vacation) # Результат: False
```

Пример из жизни для нескольких условий

Использование not с другими операторами

- not have_work → превращает True в False (у вас есть работа, значит, вы не можете поехать).
- and enough_money → проверяет, достаточно ли денег (результат: False, так как денег недостаточно).
- Окончательный результат: False, вы не можете поехать в отпуск.

```
have_work = True
```

```
enough_money = True
```

```
can_go_vacation = not have_work and enough_money
```

```
print(can_go_vacation) # Результат: False
```


Экспресс-опрос

?

Что такое таблица истинности?

?

Что такое интерпретация логических операторов?



Таблица истинности

Показывает результат работы логических операторов для всех возможных комбинаций значений True и False.

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True



Интерпретация логических операторов

В Python логические операторы `and`, `or` и `not` можно сопоставить с арифметическими операциями.

A	B	A and B ($A * B$)	A or B ($A + B$)	not A (инверсия A)
True	True	True ($1 * 1$)	True ($1 + 1$)	False ($1 \rightarrow 0$)
True	False	False ($1 * 0$)	True ($1 + 0$)	False ($1 \rightarrow 0$)
False	True	False ($0 * 1$)	True ($0 + 1$)	True ($0 \rightarrow 1$)
False	False	False ($0 * 0$)	False ($0 + 0$)	True ($0 \rightarrow 1$)

Экспресс-опрос

?

Что такое двойные неравенства?

?

Какой синтаксис двойных неравенств?



Двойные неравенства

В Python можно использовать двойные неравенства, что позволяет проверять, попадает ли число в определённый диапазон значений. Это делает код более читабельным и удобным, поскольку не требует явного использования логических операторов `and`.

Синтаксис двойных неравенств:



`min_value < variable < max_value`

Синтаксис двойных неравенств



Проверка двух условий

1. $1 < x$ — проверка, что x больше 1.
2. $x \leq 10$ — проверка, что x меньше или равно 10.

Пример

$x = 5$

```
print("1 < x <= 10") # x находится между 1 и 10
(включительно)
```

Эквивалент без двойного неравенства



Объяснение

Использование двойных неравенств эквивалентно коду с логическим оператором `and`.

Пример

```
x = 5
```

```
print(1 < x and x <= 10) # x находится между 1 и 10 (включительно)
```



ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикума.



Задание 1



Напишите программу, которая запрашивает у пользователя его имя, возраст и любимый цвет, а затем выводит краткую анкету в формате:

"Меня зовут [имя], мне [возраст] лет, и мой любимый цвет — [цвет]."

Пример вывода:

Введите имя: Анна

Введите возраст: 22

Введите любимый цвет: синий

Меня зовут Анна, мне 22 лет, и мой любимый цвет — синий.

Задание 2



Напишите программу, которая выводит на экран следующую строку несколькими способами:
Она сказала: "Привет!"

Пример вывода:

Она сказала: "Привет!"

Она сказала: "Привет!"

Она сказала: "Привет!"

Задание 3



Напишите программу, которая выводит следующий текст:

Пример вывода:

Список дел:

Учеба

Уборка

Спорт

Задание 4

Напишите программу, которая запрашивает расстояние (в километрах) и среднюю скорость автомобиля (в км/ч). Программа должна вывести время, которое потребуется, чтобы преодолеть указанное расстояние.

Пример вывода:

Введите расстояние (в км): 150

Введите среднюю скорость (в км/ч): 50

Время в пути: 3.0 часов

Задание 4.1

Доработайте программу, чтобы она выводила время, которое потребуется, чтобы преодолеть указанное расстояние в часах и минутах (без дробной части).

Пример вывода:

Введите расстояние (в км): 400

Введите среднюю скорость (в км/ч): 123

Время в пути: 3 часа и 15 минут

Задание 5



Напишите программу, которая принимает от пользователя количество дней до события и выводит, сколько это недель и дней.

Пример вывода:

Введите количество дней до события: 45

До события осталось: 6 недель и 3 дня.

Задание 6

Напишите программу, которая принимает расстояние в километрах, расход бензина на 100 км и цену за литр бензина. Программа должна рассчитать стоимость бензина для поездки.

Пример вывода:

Введите расстояние (в км): 300

Введите расход бензина на 100 км: 8

Введите цену за литр бензина: 60

Стоимость бензина для поездки: 1440.0

Задание 7



Напишите программу, которая запрашивает у пользователя количество задач и среднее время выполнения одной задачи (в минутах). Программа должна вывести общее время выполнения всех задач в формате часов и минут.

Пример вывода:

Введите количество задач: 5

Введите среднее время выполнения одной задачи (мин): 40

Общее время: 3 часа и 20 минут

Задание 8

Напишите программу, которая считывает число "n" и вычисляет выражение $n + nn + nnn$, где "nn" — это число "n", записанное дважды, а "nnn" — трижды.

Пример вывода:

Введите число: 5

Значение выражения: 615

Задание 9



Напишите программу, которая считывает четырёхзначное число и меняет местами первую и последнюю цифры.

Пример вывода:

Введите четырёхзначное число: 1234

Число после изменения: 4231



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Калькулятор поездки

Напишите программу, которая получает от пользователя расстояние (в км), расход топлива (на 100 км) и цену топлива за литр (в евро), а выводит сколько литров топлива понадобится, и сколько будет стоить поездка.

Пример ввода:

Введите расстояние (км): 500

Введите расход топлива (л на 100 км): 8

Введите цену топлива за литр: 1.8

Пример вывода:

Для поездки потребуется топлива: 40.0

Стоимость поездки: 72.0

Домашнее задание

2. Размен суммы купюрами

Напишите программу, которая получает два числа от пользователя, умножает одно число на второе и выводит результат и оба числа через дефис. Не сохраняете результат умножения в переменной.

Пример ввода:

Введите сумму: 135

Введите номинал купюры: 50

Пример вывода:

Купюр нужно: 2

Остаток: 35



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Проверка условий

Напишите программу, которая принимает на вход логические значения двух переменных (свет включён и окно открыто) и проверяет:

- Оба ли условия выполнены.
- Хотя бы одно из условий выполнено.

Пример вывода:

Свет включён? True

Окно открыто? False

Оба условия выполнены? False

Хотя бы одно условие выполнено? True

Домашнее задание

2. * Стоимость мобильного тарифа

Напишите программу для расчёта стоимости использования мобильного тарифа:

- Базовая стоимость: 30 евро.
- Каждый мегабайт интернета сверх 500 МБ стоит 0.2 евро.

Программа должна запросить у пользователя количество использованных мегабайтов и вывести сколько всего он заплатил (базовый и переплата).

Пример вывода:

Введите использованные мегабайты: 510

Общая стоимость: 32.0



ВОПРОСЫ



Заключение

